UNIVERSITÉ D'ARTOIS

TD n°5

Langage C

2026-2027

Exercice 1 :

— En utilisant les opérations bit à bit, écrire une fonction **récursive** en C qui prend en paramètre un entier positif nb et retourne le nombre de bits égaux à 1 dans sa représentation binaire. Par exemple, pour l'entier passé en paramètre 14, qui s'écrit 1110 en binaire, la fonction doit retourner 3.

— Reprendre l'exercice sans utiliser d'opérateurs if.

Exercice 2 :

En utilisant les opérations bit à bit, écrire une fonction **itérative** en C qui prend en paramètre un entier positif nb et retourne le nombre de bits égaux à 1 dans sa représentation binaire. Par exemple, pour l'entier passé en paramètre 14, qui s'écrit 1110 en binaire, la fonction doit retourner 3.

Exercice 3 :

Dans cet exercice, on suppose que nous avons n individus, identifiés simplement par les nombres $\{0, ..., n-1\}$. On s'intéresse à determiner s'il existe un agent secret parmi ces individus. Un agent secret est défini comme quelqu'un qui n'est connu par personne (sauf par lui même bien sûr) mais qui connaît tout le monde. Supposons que vous disposez d'une fonction booléenne, appelée connaît(i,j), qui retourne vraie si la personne i connaît la personne j. Supposons qu'il existe au plus un agent secret parmi les n individus.

— Ecrire une fonction (avec deux boucles imbriquées) qui retourne l'identifiant (le numéro) de l'individu agent secret (s'il existe).

— Proposer une version optimisée `cherche_agent_efficace(n)` qui trouve l'agent secret en une complexité linéaire $\mathcal{O}(n)$, c'est-à-dire en n'utilisant que des boucles simples (non imbriquées).

Exercice 4 :

Ecrire une fonction C qui prend en paramètre un entier positif n et retourne le nombre de diviseurs stricts (inférieurs à n) du nombre n. Par exemple,

— si le nombre n est égal à 12, alors la fonction retourne 5 (car le nombre 12 admet 5 diviseurs stricts 1, 2, 3, 4, 6).

Exercice 5 :

1. Ecrire une fonction itérative (non récursive) qui calcule a^b où a et b sont des entiers.
2. Ecrire une fonction récursive qui calcule a^b où a et b sont des entiers en utilisant les propriété suivantes :
   — Si $b = 0$ alors $a^b = 1$
   — Si b est pair alors $a^b = a^{(b/2)} * a^{(b/2)}$
   — Si b est impair alors $a^b = a * a^{(b-1)}$

Exercice 6 :

Écrire un programme en C qui lit un ensemble de lettres minuscules et génère tous les sous-ensembles possibles, puis les affiche.

- L'ensemble en entrée est lu caractère par caractère, commençant par '{', suivie de lettres minuscules consécutives à partir de 'a' et séparées par des virgules, se terminant par '}'.
- Pour simplifier, les lettres sont supposées se suivre de manière consécutive.
- L'utilisation de tableaux n'est pas autorisée, mais les opérateurs bit à bit peuvent être utilisés pour mémoriser et manipuler les données.
- Nous supposons que l'utilisateur respecte bien le format en entrée

Exemple :

Si l'utilisateur entre l'ensemble `{a, b, c}`, le programme devra afficher les sous-ensembles suivants (l'ordre d'affichage des sous-ensembles n'est pas pertinent) :

Sous-ensembles
{}
{a}
{b}
{c}
{a, b}
{a, c}
{b, c}
{a, b, c}

Remarque : Penser à décomposer votre programme (et votre problème).